

IIT PATNA · GENERATIVE AI CAPSTONE SPRINT 2026

KidVenture

Learn. Create. Connect.*An AI-powered digital coach for children aged 5–10*

END-TO-END BUILD MANUAL

Every step, in order — from empty account to submitted demo

Field	Detail
Project	KidVenture — AI Digital Coach for Children
Programme	IIT Patna Generative AI Capstone Sprint 2026
Group / Batch	Group 1 · Batch 5
Author	Alok Priyadarshi
Document type	End-to-end technical build manual
Version	1.0
Toolchain	OpenAI API · Lovable/Replit · Supabase · Make.com · Postman

Table of Contents

1. How to Use This Manual

This manual is written to be followed top to bottom. Each phase produces a working, testable artefact before the next phase begins. Do not skip ahead — the order exists because every later phase depends on something the earlier phase produced.

1.1 Reading conventions

Convention	Meaning
Shaded code block	Paste this exactly as written into the tool named above it.
Green box	A practical tip that saves time or cost.
Blue box	Context or a definition you need to understand before continuing.
Red box	A trap that will cost you hours if ignored. Read these twice.
☒ Done when...	The exit test for a step. If this is not true, do not move on.

1.2 The eight phases

Phase	What you build	Exit artefact
0	Accounts, keys, project skeleton	All five tool logins working
1	Supabase database, security, storage	Nine tables with RLS enabled
2	AI prompts proven in Postman	Two prompts returning valid JSON
3	Edge Functions (server logic)	Three deployed HTTP endpoints
4	Frontend in Lovable	Three working screens
5	Make.com weekly automation	A digest email in the inbox
6	Testing and hardening	Completed test checklist
7	Demo video and submission	Video, Notion page, posts

Build discipline that actually matters

Get one complete vertical slice working before you make anything look good. A plain HTML page that generates a real mission and stores a real evaluation is worth more than a beautiful UI with fake data. Commit or snapshot after every phase. Lovable and Supabase both let you roll back — use it.

2. Project Overview

2.1 The problem

- Children aged 5–10 spend increasing time on passive screen consumption — video, scrolling, games with no developmental payoff.
- Most learning apps target academics only. They do not build communication, kindness, teamwork, empathy or creative confidence.
- Parents have no simple, trustworthy way to see how a child is actually developing across cognitive and interpersonal dimensions.
- Generic content ignores that every child differs in age, interest and pace.

2.2 The solution

KidVenture turns screen time into short, purposeful daily missions. Each day a child receives up to three 15-minute missions — one per pillar:

Pillar	Focus	Example mission	Child output
LEARN	Mental maths, memory, logic, patterns, focus	Solve three number riddles to open the treasure chest	Typed answer + reasoning
CREATE	Imagination, problem-solving, storytelling	Invent a new animal and tell us what it eats	Short story or drawing photo
CONNECT	Kindness, teamwork, empathy, communication	Do one kind act today and tell us how it felt	Spoken or typed reflection

An AI evaluator reads the child's response, returns warm age-appropriate feedback, updates a skill profile, awards badges, and adjusts tomorrow's difficulty. Every Sunday a separate automation emails the parent a plain-language progress summary.

2.3 MVP scope — what is in and what is deliberately out

In scope (build this)	Out of scope (say this in the demo)
One age band: 8–10 years	Full 5–7 band with audio-first missions
Three pillars, one mission each per day	Multi-child families and sibling leaderboards
Text answers + optional photo upload	Voice recording and speech-to-text
AI evaluation and adaptive difficulty	Native mobile apps (iOS / Android)
Badges and streaks	Payments, subscriptions, school accounts
Parent dashboard + weekly email	Multi-language missions

Scope is the number one reason capstones fail

The poster lists six MVP features. Build three missions working end-to-end before you add anything else. Judges score a working narrow product far higher than a broad broken one.

Write the out-of-scope list into your demo script. Naming what you deliberately deferred reads as engineering maturity, not as a gap.

2.4 Success metric

State this single measurable outcome in your 1-pager and repeat it in the demo:

Primary success metric

A child completes a 15-minute mission, receives AI feedback within 8 seconds, and the parent dashboard reflects an updated skill score and a new developmental insight — with no manual intervention anywhere in the chain.

3. Toolchain and Prerequisites

3.1 The five tools and what each one is responsible for

Tool	Role in KidVenture	Why not something else
OpenAI API	Generates missions, screens submissions for safety, evaluates answers, writes parent summaries	Only component that produces adaptive content; JSON mode makes output parseable
Supabase	Postgres database, authentication, file storage, and serverless Edge Functions	One tool covers four needs; Row-Level Security is essential for children's data
Lovable / Replit	Child app, onboarding flow and parent dashboard	Prompt-to-app speed; you have days, not months
Make.com	Weekly scheduled job that builds and sends the parent digest	Scheduled work must run when nobody has the app open
Postman	Proving and documenting every API call before it is wired into code	Isolates AI failures from UI failures — the single biggest time saver

3.2 Accounts to create (Phase 0 checklist)

1. **OpenAI Platform account** — add a payment method and set a hard monthly spend limit of USD 10. Create a project-scoped API key named `kidventure-dev`.
2. **Supabase account** — create a new project named `kidventure`. Choose the region closest to you. Save the database password immediately; it is shown only once.
3. **Lovable (or Replit) account** — free tier is sufficient for the sprint.
4. **Make.com account** — free tier allows 1,000 operations per month, which is far more than this project needs.
5. **Postman account** — create a workspace named `KidVenture`.
6. **Notion page** — this is your submission home. Create it now and add to it as you go, not at the end.

3.3 Keys you will collect and where each one may be used

Key / value	Found in	Safe to use in browser?
Project URL	Supabase → Settings → API	Yes
anon public key	Supabase → Settings → API	Yes — RLS protects the data
service_role key	Supabase → Settings → API	NO — server-side only, ever
Database password	Set at project creation	No
OpenAI API key	OpenAI → API keys	NO — Edge Functions only

The mistake that ends capstone projects early

Never put the OpenAI key or the service_role key into Lovable frontend code, a public GitHub repo, or a screenshot in your demo video. Both keys bypass all security. If either leaks, rotate it immediately from the provider console.

Every AI call in this design goes through a Supabase Edge Function for exactly this reason.

4. Architecture

Three views of the same system. Read them in order: the layer view shows what exists, the sequence view shows what happens, the data view shows what is stored.

4.1 System architecture — the layer view

KidVenture — End-to-End System Architecture

Presentation, Application, Intelligence, Data and Automation layers with every connection

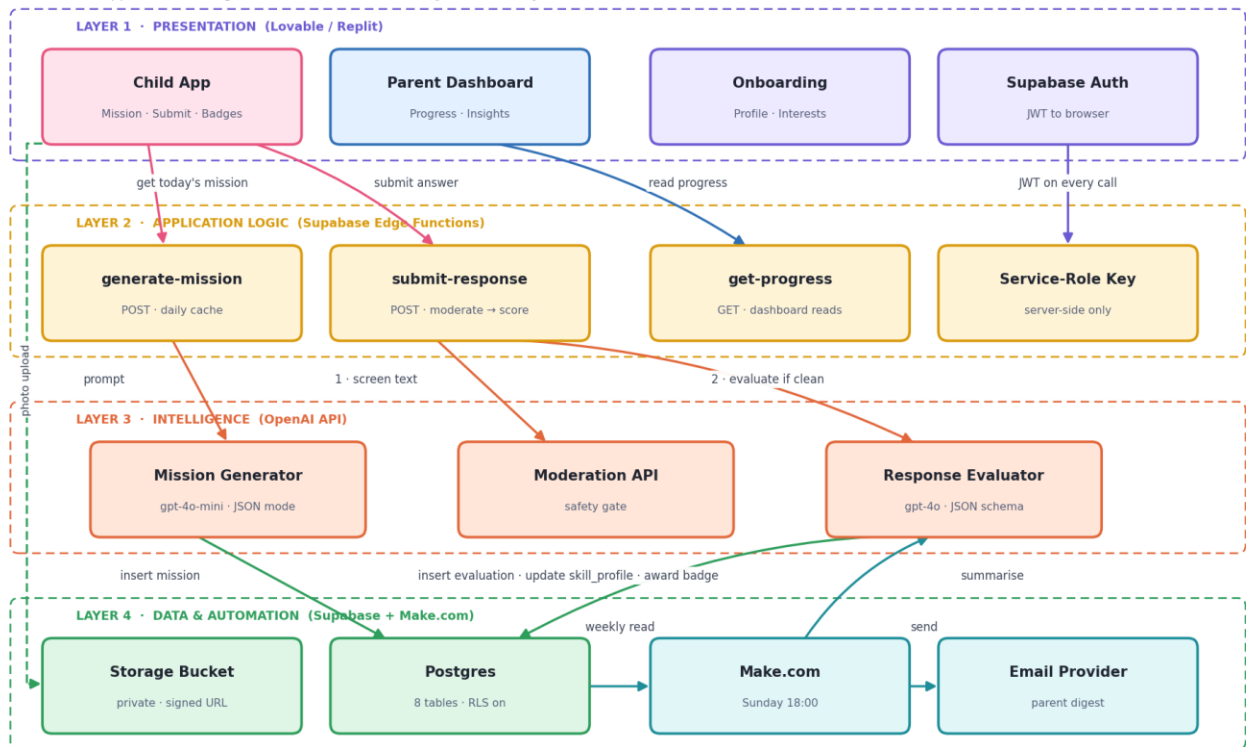


Figure 1 — Five-layer architecture showing every service and connection

Read it top to bottom:

- **Layer 1 — Presentation.** Everything the user touches, built in Lovable. It holds no secrets and contains no business logic. It authenticates with Supabase Auth and receives a JWT that is attached to every subsequent request.
- **Layer 2 — Application logic.** Three Supabase Edge Functions. These are the only components that hold the OpenAI key and the service-role key. All validation, ordering and transaction logic lives here.
- **Layer 3 — Intelligence.** Three distinct OpenAI calls with three distinct jobs. The Moderation API sits between the child's input and the evaluator — it is a gate, not a logger.
- **Layer 4 — Data and automation.** Postgres holds all state with Row-Level Security enforced. Storage holds photos in a private bucket. Make.com reads the week's data on a schedule and sends the parent digest.

Note the single dashed line down the left edge: photo uploads go from the browser straight to Supabase Storage using a short-lived signed URL. They never pass through an Edge Function, which keeps function execution fast and cheap.

4.2 Core loop sequence — what actually happens

KidVenture — Core Loop Sequence (Mission → Submission → Adaptation)

Numbered end-to-end call sequence across all five actors

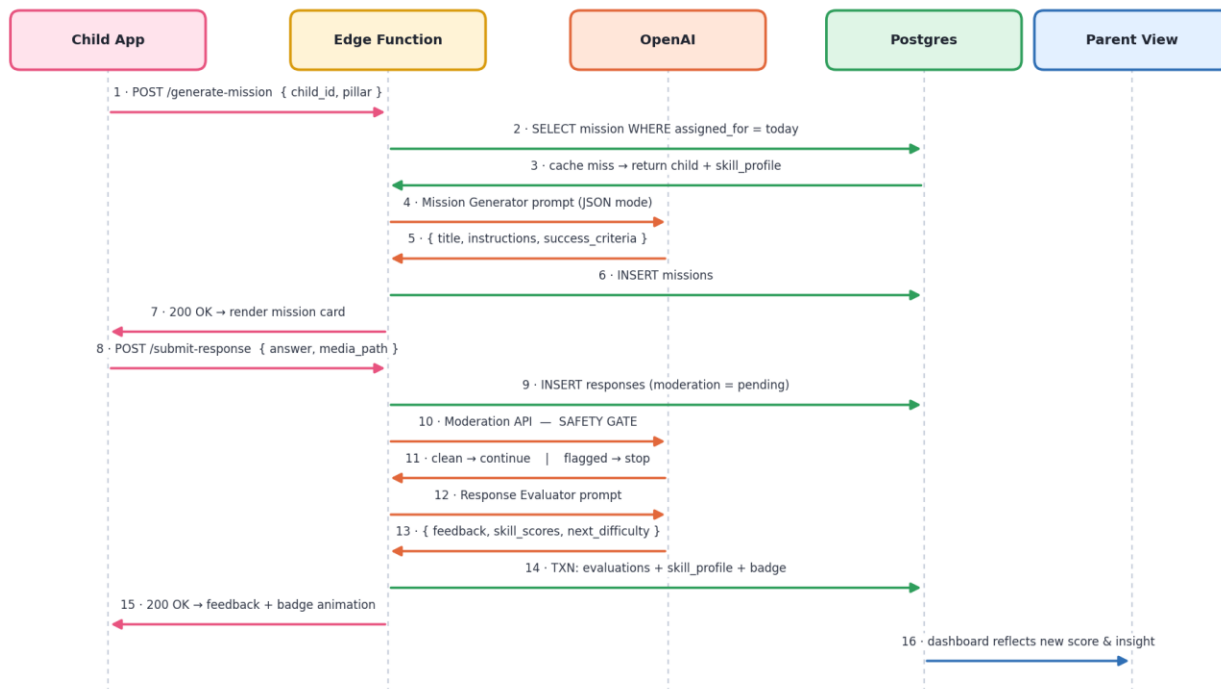


Figure 2 — The sixteen-step mission and evaluation loop

Two things in this sequence deserve attention because they are where most implementations go wrong.

Step 2 — the daily cache check. Before any AI call, the function checks whether a mission already exists for this child, this pillar, today. A unique database index enforces it. Without this, every page refresh burns an OpenAI call and the child sees a different mission each time they reload.

Steps 10 and 11 — the safety gate. Moderation runs *before* the evaluator ever sees the text. If the content is flagged, the flow stops, the child gets a gentle retry message, and no evaluation is written. Ordering is what makes this a real control rather than a claim on a slide.

4.3 Data model — what is stored

KidVenture — Data Model (Entity Relationships)

Reference data in green · event data in amber/orange · gamification in purple · automation in teal

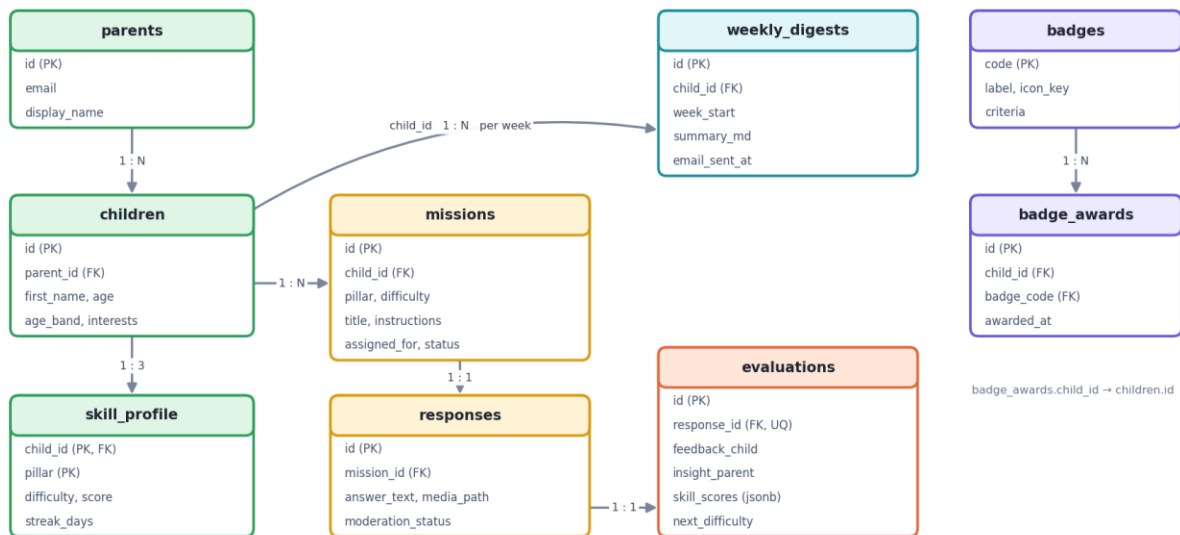


Figure 3 — Nine tables across reference, event, gamification and automation groups

Group	Tables	Characteristic
Identity & reference	parents, children, skill_profile	Current state. skill_profile is updated in place — exactly three rows per child.
Event data	missions, responses, evaluations	Append-only. Never updated after insert (except mission status). This lets you recompute scoring if the algorithm changes mid-build.
Gamification	badges, badge_awards	badges is static seed data. badge_awards has a unique constraint so a badge can never be awarded twice.
Automation	weekly_digests	One row per child per week, so a re-run of the Make.com scenario cannot double-send.

Why skill_profile is separate from evaluations

evaluations records what the AI thought about one specific answer. skill_profile records where the child stands right now. Keeping them apart means you can change how scores are calculated without losing any history — you simply replay the evaluations.

5. Phase 1 — Supabase Foundation

Target: 4–6 hours. You will finish with a complete, secured database and a private storage bucket.

5.1 Create the project

1. Sign in to Supabase and select New Project.
2. Name it kidventure, set a strong database password, and pick your nearest region.
3. Wait for provisioning to complete (about two minutes).
4. Go to Settings → API and copy the Project URL, the anon key and the service_role key into a private notes file.

☒ **Done when: you can open the SQL Editor and run SELECT 1; successfully.**

5.2 Create the tables

Open SQL Editor → New query. Paste the whole script below and run it once. It is written to run top to bottom without errors because the table order respects the foreign keys.

```
-- =====
-- KidVenture · schema v1
-- =====

-- 1. PARENTS (anchored to Supabase Auth)
create table parents (
  id          uuid primary key references auth.users(id) on delete cascade,
  email       text not null unique,
  display_name text,
  created_at  timestampz default now()
);

-- 2. CHILDREN (minimal personal data by design)
create table children (
  id          uuid primary key default gen_random_uuid(),
  parent_id   uuid not null references parents(id) on delete cascade,
  first_name  text not null,
  age         int not null check (age between 5 and 10),
  age_band    text generated always as
              (case when age <= 7 then '5-7' else '8-10' end) stored,
  interests   text[] default '{}',
  avatar_key  text,
  created_at  timestampz default now()
);
create index idx_children_parent on children(parent_id);

-- 3. SKILL PROFILE (current state: exactly 3 rows per child)
create table skill_profile (
  child_id    uuid not null references children(id) on delete cascade,
  pillar       text not null check (pillar in ('learn','create','connect')),
  difficulty   int not null default 2 check (difficulty between 1 and 5),
  score        numeric(6,2) not null default 0,
  streak_days  int not null default 0,
  last_mission_at timestampz,
  updated_at   timestampz default now(),
  primary key (child_id, pillar)
);

-- 4. BADGES (static seed data)
create table badges (
  code        text primary key,
  label       text not null,
```

```

    icon_key text,
    pillar text,
    criteria text not null
);

-- 5. MISSIONS (AI generated, cached one per pillar per day)
create table missions (
  id          uuid primary key default gen_random_uuid(),
  child_id    uuid not null references children(id) on delete cascade,
  pillar      text not null check (pillar in ('learn','create','connect')),
  age_band    text not null,
  difficulty  int not null check (difficulty between 1 and 5),
  title       text not null,
  instructions text not null,
  success_criteria text not null,
  input_type  text not null default 'text'
              check (input_type in ('text','photo','choice')),
  est_minutes int default 15,
  status      text not null default 'assigned'
              check (status in ('assigned','completed','skipped','expired')),
  assigned_for date not null default current_date,
  model_meta  jsonb,
  created_at  timestampz default now()
);
create unique index one_mission_per_pillar_per_day
  on missions(child_id, pillar, assigned_for);

-- 6. RESPONSES (child submissions)
create table responses (
  id          uuid primary key default gen_random_uuid(),
  mission_id  uuid not null references missions(id) on delete cascade,
  child_id    uuid not null references children(id) on delete cascade,
  answer_text text,
  media_path  text,
  time_spent_sec int,
  moderation_status text not null default 'pending'
              check (moderation_status in ('pending','clean','flagged')),
  moderation_meta jsonb,
  submitted_at timestampz default now()
);
create index idx_responses_child on responses(child_id, submitted_at desc);

-- 7. EVALUATIONS (AI feedback, exactly one per response)
create table evaluations (
  id          uuid primary key default gen_random_uuid(),
  response_id uuid not null unique references responses(id) on delete cascade,
  child_id    uuid not null references children(id) on delete cascade,
  pillar      text not null,
  feedback_child text not null,
  insight_parent text not null,
  skill_scores jsonb not null,
  effort_score int check (effort_score between 1 and 5),
  next_difficulty int not null check (next_difficulty between 1 and 5),
  badge_code  text references badges(code),
  created_at  timestampz default now()
);

-- 8. BADGE AWARDS
create table badge_awards (
  id          uuid primary key default gen_random_uuid(),
  child_id    uuid not null references children(id) on delete cascade,
  badge_code  text not null references badges(code),
  awarded_at  timestampz default now(),
  unique (child_id, badge_code)
);

-- 9. WEEKLY DIGESTS (Make.com output log)
create table weekly_digests (

```

```

id          uuid primary key default gen_random_uuid(),
child_id    uuid not null references children(id) on delete cascade,
week_start  date not null,
summary_md  text not null,
metrics     jsonb,
email_sent_at timestamptz,
unique (child_id, week_start)
);

```

Script 1 — Run once in Supabase SQL Editor. Creates all nine tables.

☒ **Done when: the Table Editor lists nine tables and no errors appear in the SQL output panel.**

5.3 Seed the badges

```

insert into badges (code, label, icon_key, pillar, criteria) values
('first_steps', 'First Steps', 'star', null, 'Complete your very first mission'),
('streak_3', '3-Day Explorer', 'flame', null, 'Complete a mission 3 days in a row'),
('streak_7', 'Week Champion', 'trophy', null, 'Complete a mission 7 days in a row'),
('logic_master', 'Logic Master', 'brain', 'learn', 'Score 4+ on five LEARN missions'),
('story_weaver', 'Story Weaver', 'book', 'create', 'Complete three storytelling missions'),
('kind_heart', 'Kind Heart', 'heart', 'connect', 'Complete three kindness missions'),
('level_up', 'Level Up', 'rocket', null, 'Reach difficulty 4 in any pillar');

```

Script 2 — Seed data. Safe to re-run after deleting rows.

5.4 Turn on Row-Level Security

Do this before you write a single line of frontend code

This application stores data about minors. Until RLS is enabled, your anon key allows anyone with the URL to read every child record in the database. Enabling it later means retro-fitting policies while debugging a UI — do it now.

```

-- Enable RLS on every table holding user data
alter table parents      enable row level security;
alter table children     enable row level security;
alter table skill_profile enable row level security;
alter table missions     enable row level security;
alter table responses    enable row level security;
alter table evaluations   enable row level security;
alter table badge_awards enable row level security;
alter table weekly_digests enable row level security;

-- A parent can see only their own record
create policy parent_self on parents
  for all using (id = auth.uid());

-- A parent can see only their own children
create policy parent_owns_child on children
  for all using (parent_id = auth.uid());

-- Helper: does the current user own this child?
create or replace function owns_child(cid uuid) returns boolean as $$
select exists (
  select 1 from children c
  where c.id = cid and c.parent_id = auth.uid()
)

```

```

);
$$ language sql security definer stable;

-- Apply the same rule to every child-scoped table
create policy p_skill    on skill_profile    for all using (owns_child(child_id));
create policy p_miss     on missions        for all using (owns_child(child_id));
create policy p_resp     on responses       for all using (owns_child(child_id));
create policy p_eval     on evaluations     for all using (owns_child(child_id));
create policy p_award    on badge_awards   for all using (owns_child(child_id));
create policy p_digest   on weekly_digests for all using (owns_child(child_id));

-- badges is public reference data
alter table badges enable row level security;
create policy badges_read on badges for select using (true);

```

Script 3 — Row-Level Security. Run after Script 1.

☒ **Done when: Table Editor shows a green “RLS enabled” shield on all nine tables.**

5.5 Create the storage bucket

1. Go to Storage → New bucket. Name it kidventure-media.
2. Leave Public bucket switched **OFF**. This is not optional for a children's product.
3. Set the file size limit to 5 MB and allowed MIME types to image/jpeg, image/png.
4. Adopt this path convention for every upload: {child_id}/{mission_id}/{uuid}.jpg.

```

-- Storage policy: a parent may read and write only inside
-- folders belonging to their own child
create policy media_rw on storage.objects
  for all using (
    bucket_id = 'kidventure-media'
    and owns_child( (storage.foldername(name))[1]::uuid )
  );

```

Script 4 — Storage access policy.

The frontend never gets a public URL. It requests a signed URL valid for 60 seconds, uses it, and discards it.

6. Phase 2 — Prove the AI in Postman

Target: 3–4 hours. This is the phase teams skip, and it is the phase that saves the project. You will confirm that both prompts return valid, parseable JSON before any application code exists.

6.1 Set up the Postman workspace

1. Create a workspace called KidVenture and inside it a collection called KidVenture API.
2. Create an Environment named Dev with these variables:

Variable	Value	Type
openai_key	sk-...your key...	secret
supabase_url	https://xxxx.supabase.co	default
anon_key	eyJhbGciOi...	secret
child_id	(filled by test script)	default
mission_id	(filled by test script)	default

3. Create six folders: 00-Auth, 01-Onboarding, 02-Missions, 03-Submission, 04-Dashboard, 05-Digest.

6.2 Prompt A — the Mission Generator

Create a POST request to <https://api.openai.com/v1/chat/completions> with header Authorization: Bearer {{openai_key}}.

```
{
  "model": "gpt-4o-mini",
  "response_format": { "type": "json_object" },
  "temperature": 0.8,
  "messages": [
    {
      "role": "system",
      "content": "You design short daily missions for children aged 5 to 10 in a learning app called KidVenture. You must return ONLY a JSON object with exactly these keys: title, instructions, success_criteria, input_type, est_minutes, difficulty. RULES: (1) Write instructions directly to the child in warm, simple language at their reading level. (2) Never exceed 3 sentences of instruction. (3) The mission must be completable in about 15 minutes with no adult help and no special materials beyond paper and pencil. (4) input_type must be one of text, photo, choice. (5) difficulty must equal the requested difficulty. (6) Never reference violence, money, brands, or anything frightening. (7) LEARN missions build mental maths, memory, logic or patterns. CREATE missions build imagination, storytelling or problem-solving. CONNECT missions build kindness, empathy, teamwork or communication with a real person."
    },
    {
      "role": "user",
      "content": "Child age: 9. Age band: 8-10. Interests: dinosaurs, football, drawing. Pillar: create. Requested difficulty: 3 of 5. Recent mission titles to avoid repeating: The Dragon's Riddle, My Rainy Day Story."
    }
  ]
}
```

Request A — Mission Generator. Paste into Postman → Body → raw → JSON.

Expected response shape:

```
{
  "title": "The Dinosaur Who Played Football",
  "instructions": "Invent a dinosaur who joins a football team! Tell us its name, what position it plays, and one funny problem it causes on the field.",
  "success_criteria": "Child names the dinosaur, gives it a position, and describes one imaginative problem.",
  "input_type": "text",
  "est_minutes": 15,
  "difficulty": 3
}
```

Expected output A.

Test it properly before moving on

Run the request six times — twice per pillar. Vary difficulty between 1 and 5. Confirm that difficulty 1 missions are genuinely simpler than difficulty 5 missions, and that the language changes for age 6 versus age 10.

If two runs return near-identical missions, raise temperature to 0.9 and make sure you are passing the recent-titles list.

6.3 Prompt B — the Response Evaluator

Feedback tone is a product requirement, not a nicety

This model output is read by a seven-year-old. A discouraging sentence does real harm and will sink your demo. Build the tone rules into the system prompt and then deliberately test with a blank answer and a wrong answer to confirm the model never shames the child.

```
{
  "model": "gpt-4o",
  "response_format": { "type": "json_object" },
  "temperature": 0.4,
  "messages": [
    {
      "role": "system",
      "content": "You evaluate a child's response to a KidVenture mission. Return ONLY a JSON object with exactly these keys: feedback_child, insight_parent, skill_scores, effort_score, next_difficulty, badge_suggestion. RULES: (1) feedback_child is 2 to 3 sentences written TO the child. Always name one specific thing they did well, then offer one gentle next step starting with 'Next time, try...'. Never use the words wrong, bad, failed, or incorrect. Never shame. (2) insight_parent is 1 to 2 sentences written TO the parent, describing an observed developmental behaviour rather than a grade. (3) skill_scores is an object of 2 to 4 named skills scored 1 to 5, chosen from: logic, memory, numeracy, creativity, storytelling, problem_solving, empathy, communication, teamwork. (4) effort_score is 1 to 5 and reflects effort and engagement, NOT correctness. (5) next_difficulty: raise by 1 if effort_score is 4 or 5 and the response clearly met the success criteria; lower by 1 if the child appeared to struggle or gave up; otherwise keep it the same. Stay within 1 to 5. (6) badge_suggestion is a badge code or null. (7) A blank or very short answer still receives warm encouragement and a LOWER next_difficulty, never criticism."
    },
    {
      "role": "user",
      "content": "MISSION: The Dinosaur Who Played Football | INSTRUCTIONS: Invent a dinosaur who joins a football team. | SUCCESS CRITERIA: Child names the dinosaur, gives it a position, and describes one imaginative problem. | CURRENT DIFFICULTY: 3 | CHILD AGE: 9 | CHILD ANSWER: His name is Rexy and he is the goalkeeper because his arms are too small to throw but his body is really big so nobody can score. The problem is when he gets excited he steps on the ball and it pops."
    }
  ]
}
```

Request B — Response Evaluator.

Expected response shape:

```
{
  "feedback_child": "Rexy the goalkeeper is a brilliant idea – you thought carefully about how his tiny arms and big body would actually change the way he plays! The popping ball made me laugh out loud. Next time, try adding what his teammates say when it happens.",
  "insight_parent": "Your child linked a character's physical traits to a logical consequence, which shows cause-and-effect reasoning applied inside imaginative play.",
  "skill_scores": { "creativity": 5, "storytelling": 4, "problem_solving": 4 },
  "effort_score": 5,
  "next_difficulty": 4,
  "badge_suggestion": "story_weaver"
}
```

Expected output B.

6.4 The four tests you must run before Phase 3

Test	Input	Required behaviour
Strong answer	Detailed, on-criteria response	next_difficulty increases; warm specific praise
Weak answer	"idk"	next_difficulty decreases; still encouraging; no shaming words
Blank answer	Empty string	Valid JSON returned; effort_score 1; gentle invitation to retry
Off-topic answer	Random unrelated text	Valid JSON; feedback redirects kindly to the mission

☒ **Done when:** all four tests return valid JSON that parses without error, and the tone rules hold in every case.

6.5 Prompt C — the Moderation gate

```
POST https://api.openai.com/v1/moderations
Authorization: Bearer {{openai_key}}
```

```
{
  "model": "omni-moderation-latest",
  "input": "{{child_answer_text}}"
}
```

Request C — Moderation. Read results[0].flagged in your Edge Function.

7. Phase 3 — Edge Functions (Server Logic)

Target: 6–8 hours. You will deploy three HTTP endpoints. These hold every secret and enforce every rule.

7.1 Store the secrets

In Supabase go to Edge Functions → Secrets and add:

Secret name	Value
OPENAI_API_KEY	Your OpenAI key
SUPABASE_URL	Your project URL
SERVICE_ROLE_KEY	Your service_role key

7.2 Function 1 — generate-mission

Responsibility: return today's mission for one child and one pillar, generating it only if it does not already exist.

```
// supabase/functions/generate-mission/index.ts
import { createClient } from "https://esm.sh/@supabase/supabase-js@2";

const db = createClient(
  Deno.env.get("SUPABASE_URL")!,
  Deno.env.get("SERVICE_ROLE_KEY")!
);
const OPENAI = Deno.env.get("OPENAI_API_KEY")!;

Deno.serve(async (req) => {
  const { child_id, pillar } = await req.json();
  const today = new Date().toISOString().slice(0, 10);

  // STEP 1 – cache check (stops duplicate spend on refresh)
  const { data: existing } = await db.from("missions")
    .select("*")
    .eq("child_id", child_id).eq("pillar", pillar)
    .eq("assigned_for", today).maybeSingle();
  if (existing) return Response.json(existing);

  // STEP 2 – read child context and current difficulty
  const { data: child } = await db.from("children")
    .select("age, age_band, interests").eq("id", child_id).single();
  const { data: prof } = await db.from("skill_profile")
    .select("difficulty").eq("child_id", child_id).eq("pillar", pillar).single();

  // STEP 3 – avoid repeating recent missions
  const { data: recent } = await db.from("missions")
    .select("title").eq("child_id", child_id).eq("pillar", pillar)
    .order("created_at", { ascending: false }).limit(5);
  const avoid = (recent ?? []).map(r => r.title).join(", ");

  // STEP 4 – call OpenAI (system prompt from section 6.2)
  const ai = await fetch("https://api.openai.com/v1/chat/completions", {
    method: "POST",
    headers: { "Authorization": `Bearer ${OPENAI}`, "Content-Type": "application/json" },
    body: JSON.stringify({
```

```

    model: "gpt-4o-mini",
    response_format: { type: "json_object" },
    temperature: 0.8,
    messages: [
      { role: "system", content: MISSION_SYSTEM_PROMPT },
      { role: "user", content:
        `Child age: ${child.age}. Age band: ${child.age_band}. ` +
        `Interests: ${child.interests.join(", ")}. Pillar: ${pillar}. ` +
        `Requested difficulty: ${prof.difficulty} of 5. ` +
        `Recent mission titles to avoid repeating: ${avoid}.` }
    ]
  })
});
const json = await ai.json();
const m = JSON.parse(json.choices[0].message.content);

// STEP 5 – persist and return
const { data: saved, error } = await db.from("missions").insert({
  child_id, pillar,
  age_band: child.age_band,
  difficulty: prof.difficulty,
  title: m.title,
  instructions: m.instructions,
  success_criteria: m.success_criteria,
  input_type: m.input_type,
  est_minutes: m.est_minutes,
  assigned_for: today,
  model_meta: { model: "gpt-4o-mini", tokens: json.usage?.total_tokens }
}).select().single();

if (error) return Response.json({ error: error.message }, { status: 500 });
return Response.json(saved);
});

```

Function 1 — generate-mission.

7.3 Function 2 — submit-response

Responsibility: the entire core loop. Moderate, evaluate, persist, adapt, award — in that exact order.

```

// supabase/functions/submit-response/index.ts

Deno.serve(async (req) => {
  const { mission_id, child_id, answer_text, media_path,
    time_spent_sec } = await req.json();

  // STEP 1 – record the raw submission first
  const { data: resp } = await db.from("responses").insert({
    mission_id, child_id, answer_text, media_path, time_spent_sec,
    moderation_status: "pending"
  }).select().single();

  // STEP 2 – SAFETY GATE (must run before the evaluator)
  const mod = await fetch("https://api.openai.com/v1/moderations", {
    method: "POST",
    headers: { "Authorization": `Bearer ${OPENAI}`,
      "Content-Type": "application/json" },
    body: JSON.stringify({ model: "omni-moderation-latest",
      input: answer_text ?? "" })
  }).then(r => r.json());

  if (mod.results[0].flagged) {
    await db.from("responses").update({
      moderation_status: "flagged",
      moderation_meta: mod.results[0].categories
    }).eq("id", resp.id);
  }
});

```

```

    return Response.json({
      blocked: true,
      message: "Let's try writing that a different way!"
    });
  }
  await db.from("responses")
    .update({ moderation_status: "clean" }).eq("id", resp.id);

  // STEP 3 – evaluate (system prompt from section 6.3)
  const { data: mission } = await db.from("missions")
    .select("*").eq("id", mission_id).single();
  const { data: child } = await db.from("children")
    .select("age").eq("id", child_id).single();

  const ev = await callOpenAI(EVALUATOR_SYSTEM_PROMPT,
    `MISSION: ${mission.title} | INSTRUCTIONS: ${mission.instructions} | ` +
    `SUCCESS CRITERIA: ${mission.success_criteria} | ` +
    `CURRENT DIFFICULTY: ${mission.difficulty} | CHILD AGE: ${child.age} | ` +
    `CHILD ANSWER: ${answer_text ?? "(no answer given)"}`);

  // STEP 4 – persist evaluation
  await db.from("evaluations").insert({
    response_id: resp.id, child_id, pillar: mission.pillar,
    feedback_child: ev.feedback_child,
    insight_parent: ev.insight_parent,
    skill_scores: ev.skill_scores,
    effort_score: ev.effort_score,
    next_difficulty: ev.next_difficulty,
    badge_code: ev.badge_suggestion
  });

  // STEP 5 – adapt the skill profile (streak logic included)
  const { data: prof } = await db.from("skill_profile").select("*")
    .eq("child_id", child_id).eq("pillar", mission.pillar).single();

  const last = prof.last_mission_at ? new Date(prof.last_mission_at) : null;
  const oneDay = 86400000;
  const continued = last &&
    (Date.now() - last.getTime()) < 2 * oneDay;

  await db.from("skill_profile").update({
    difficulty: ev.next_difficulty,
    score: Number(prof.score) + ev.effort_score,
    streak_days: continued ? prof.streak_days + 1 : 1,
    last_mission_at: new Date().toISOString(),
    updated_at: new Date().toISOString()
  }).eq("child_id", child_id).eq("pillar", mission.pillar);

  // STEP 6 – award badge (unique constraint makes this idempotent)
  if (ev.badge_suggestion) {
    await db.from("badge_awards")
      .upsert({ child_id, badge_code: ev.badge_suggestion },
        { onConflict: "child_id,badge_code", ignoreDuplicates: true });
  }

  // STEP 7 – close the mission
  await db.from("missions")
    .update({ status: "completed" }).eq("id", mission_id);

  return Response.json({
    feedback: ev.feedback_child,
    badge: ev.badge_suggestion,
    new_difficulty: ev.next_difficulty
  });
});

```

Function 2 — submit-response. This is the heart of the product.

Make steps 4 to 7 atomic if you have time

If a failure occurs between step 4 and step 5, the child has an evaluation but an unchanged difficulty. Wrapping steps 4–7 in a Postgres function called via `db.rpc()` makes the whole update succeed or fail together. Treat this as a stretch goal — the sequence above works for the demo.

7.4 Function 3 — get-progress

Responsibility: one read call that returns everything the parent dashboard needs, so the frontend makes a single request rather than five.

```
// Returns: { child, pillars[], badges[], recent_insights[], latest_digest }
Deno.serve(async (req) => {
  const child_id = new URL(req.url).searchParams.get("child_id")!;

  const [child, pillars, awards, insights, digest] = await Promise.all([
    db.from("children").select("first_name, age, interests")
      .eq("id", child_id).single(),
    db.from("skill_profile").select("*").eq("child_id", child_id),
    db.from("badge_awards").select("badge_code, awarded_at, badges(label, icon_key)")
      .eq("child_id", child_id),
    db.from("evaluations").select("insight_parent, pillar, created_at")
      .eq("child_id", child_id)
      .order("created_at", { ascending: false }).limit(5),
    db.from("weekly_digests").select("summary_md, week_start")
      .eq("child_id", child_id)
      .order("week_start", { ascending: false }).limit(1).maybeSingle()
  ]);

  return Response.json({
    child: child.data, pillars: pillars.data, badges: awards.data,
    recent_insights: insights.data, latest_digest: digest.data
  });
});
```

Function 3 — get-progress.

7.5 Deploy and verify

1. Deploy each function from the Supabase dashboard or CLI.
2. Add each endpoint to your Postman collection in the matching folder.
3. Chain the variables: in the onboarding request's Tests tab add `pm.environment.set("child_id", pm.response.json().id)` so the whole collection runs top to bottom in one click.

☒ **Done when: running the full Postman collection produces a mission, a submission, an evaluation, an updated skill_profile row and a badge — with no manual editing between requests.**

8. Phase 4 — Frontend in Lovable

Target: 8–10 hours. Three screens. Resist every temptation to add a fourth.

8.1 Screen 1 — Onboarding

Element	Detail
Fields	Child first name, age (5–10 slider), pick 3 interests from 12 icon tiles
On submit	Insert into children, then insert three skill_profile rows at difficulty 2
Visual	Full-screen friendly illustration, one question at a time, very large tap targets
Guardrail	Collect first name and age only. No surname, no school, no photo of the child.

8.2 Screen 2 — Mission screen (the one that must be flawless)

State	What the child sees
Loading	Animated mascot with “Finding today's adventure...” — never a blank spinner
Mission card	Big title, 2–3 line instructions, pillar colour band, estimated time
Input	Large text area or camera/upload button, depending on input_type
Submitting	Mascot “thinking” animation while the evaluator runs (3–8 seconds)
Feedback	feedback_child in large friendly type, badge animation if awarded
Blocked	Gentle “Let's try writing that a different way!” with the input preserved

Lovable build prompt

```
Build a mobile-first screen for children aged 8-10.

Top: a rounded mission card with a large bold title, 2-3 lines of
instructions, a coloured band indicating the pillar (LEARN = blue,
CREATE = purple, CONNECT = pink), and an estimated time chip.

Middle: a large rounded text area with placeholder "Type your answer
here!" OR a big camera button when input_type is photo.

Bottom: a full-width rounded submit button labelled "I'm done!".

After submit: replace the card with a feedback panel showing the AI
feedback in large friendly type, plus a badge that scales in with a
bounce animation if one was awarded.

Style: playful, high contrast, minimum 18px body text, generous
padding, rounded corners, no dense text anywhere.
```

Paste into Lovable to generate Screen 2.

8.3 Screen 3 — Parent dashboard

Section	Content	Source field
Header	Child name, age, current streak	children + skill_profile
Three skill bars	LEARN / CREATE / CONNECT score and difficulty level	skill_profile
Insights feed	Last five developmental observations	evaluations.insight_parent
Badge shelf	Earned badges in colour, unearned greyed out	badge_awards + badges
Weekly summary	Most recent digest text	weekly_digests.summary_md

The dashboard is what converts the parent — write it for them

Parents do not want scores, they want reassurance and understanding. Lead the dashboard with the insights feed, not the numbers. “Your child linked a character's traits to a logical consequence” persuades far more than “Creativity: 4/5”.

8.4 Wiring the frontend to the backend

```
// Get today's mission
const res = await fetch(
  `${SUPABASE_URL}/functions/v1/generate-mission`,
  { method: "POST",
    headers: { Authorization: `Bearer ${session.access_token}`,
      "Content-Type": "application/json" },
    body: JSON.stringify({ child_id, pillar: "learn" }) });

// Photo upload – browser goes straight to Storage, not via a function
const path = `${child_id}/${mission_id}/${crypto.randomUUID()}.jpg`;
await supabase.storage.from("kidventure-media").upload(path, file);

// Submit the answer
await fetch(`${SUPABASE_URL}/functions/v1/submit-response`, {
  method: "POST",
  headers: { Authorization: `Bearer ${session.access_token}`,
    "Content-Type": "application/json" },
  body: JSON.stringify({ mission_id, child_id, answer_text,
    media_path: path, time_spent_sec })
});
```

Frontend calls. Only the anon key and the user JWT ever appear here.

☒ **Done when: a real child profile can complete a real mission on a phone-sized screen and see real AI feedback.**

9. Phase 5 — Make.com Weekly Digest

Target: 2–3 hours. One scenario, six modules. This demonstrates the automation tool category and produces a tangible artefact you can show landing in an inbox.

#	Module	Configuration
1	Schedule trigger	Every Sunday at 18:00 local time
2	Supabase — search rows	Table children; returns every active child
3	Iterator	Loop once per child
4	Supabase — search rows	Table evaluations; filter child_id = current and created_at within last 7 days
5	OpenAI — create completion	Digest prompt (below); output is markdown
6	Supabase — create a row	Insert into weekly_digests
7	Email — send	To the parent email; then update email_sent_at

9.1 The digest prompt

SYSTEM:

You write a warm weekly progress note to a parent about their child's week in KidVenture. Use plain everyday language with no jargon and no scores or percentages. Structure: (1) one opening sentence naming something specific and genuine the child did well this week, (2) three short bullets – one per pillar – describing an observed behaviour, not a grade, (3) one closing suggestion of a simple offline activity the parent can do with the child this week. Maximum 160 words. Never use the words score, level, rank, or performance.

USER:

Child first name: {{first_name}}
 Age: {{age}}
 Missions completed this week: {{count}}
 Current streak: {{streak_days}} days
 Evaluator observations this week:
 {{insight_parent list from module 4}}

Module 5 prompt configuration.

Demo-day trick

Keep a manual “Run once” button ready in Make.com. Nobody wants to wait until Sunday during your demo — trigger it live and let the email arrive on screen. It is the single most convincing moment in the whole walkthrough.

☒ **Done when:** a real digest email arrives in your inbox and the matching weekly_digests row has a populated email_sent_at.

10. Phase 6 — Testing and Hardening

10.1 Functional test checklist

#	Test	Pass criteria
1	Create a child profile	Three skill_profile rows appear at difficulty 2
2	Load a mission	Mission is age-appropriate and references an interest
3	Refresh the page	The same mission returns; no new OpenAI call is made
4	Submit a strong answer	Difficulty rises by 1; feedback is specific and warm
5	Submit “idk”	Difficulty drops by 1; no shaming language anywhere
6	Submit a blank answer	Valid response; child is gently invited to try again
7	Submit unsafe text	Blocked before evaluation; response marked flagged
8	Upload a photo	File lands in the private bucket under the correct path
9	Complete missions two days running	streak_days becomes 2
10	Earn the same badge twice	Only one badge_awards row exists
11	Open the parent dashboard	Scores, insights and badges all reflect reality
12	Run the Make.com scenario	Email received; digest row written

10.2 Security test checklist

#	Test	Pass criteria
1	Query children with a second account's token	Zero rows returned (RLS holding)
2	Open a storage file URL without a signature	Access denied
3	Search the frontend bundle for “sk-”	No match found
4	Inspect browser network tab during a mission	No OpenAI call visible from the client

10.3 Cost and performance guardrails

Concern	Control already in the design	Target
Duplicate AI spend	Unique index caches one mission per pillar per day	≤ 4 OpenAI calls per child per day
Slow feedback	gpt-4o-mini for generation; gpt-4o only for evaluation	Feedback in under 8 seconds

Concern	Control already in the design	Target
Runaway cost	Hard monthly spend limit set on the OpenAI account	Under USD 10 for the sprint
Malformed AI output	JSON mode plus a try/catch with one retry	Zero unhandled parse failures

11. Phase 7 — Demo Video and Submission

11.1 The 5-minute demo structure

Time	Segment	What is on screen
0:00–0:30	The problem	You speaking; one slide with the four problem points
0:30–1:00	The idea	The KidVenture poster; state the three pillars
1:00–2:30	Live mission	Child app: load a mission, type an answer, submit
2:30–3:00	AI feedback	Feedback text and badge animation appearing
3:00–3:45	Adaptation proof	Submit a weak answer, show difficulty visibly drop
3:45–4:20	Parent dashboard	Skill bars, insights feed, badge shelf
4:20–4:45	Automation	Trigger Make.com live; email arrives on screen
4:45–5:00	Close	Architecture diagram plus the success metric

Three recording rules

Record with seed data already present. A brand-new empty profile makes the dashboard look broken on camera.

Never show the Supabase Settings → API page or an Edge Function secrets screen. Keys have leaked this way.

Do the weak-answer test on camera. Showing the difficulty drop is the moment “adaptive” stops being a claim.

11.2 Submission checklist

Deliverable	Where it lives	Status
1-Pager Idea Document	Notion page	☐
Architecture diagrams (3)	Notion page	☐
Working prototype link	Lovable published URL	☐
Postman collection export	Notion attachment	☐
Demo video (3–5 min)	Loom / YouTube unlisted	☐
Build-in-public posts	LinkedIn, tagged	☐
Technical documentation	Notion page	☐

11.3 Judging criteria and where this build earns marks

Criterion	How this build addresses it
Problem identification and relevance	Screen time versus development is a problem every parent recognises immediately; the four problem statements are concrete and specific.
Use of the AI toolchain	All five tools used for a genuine purpose — not decoratively. Postman evidence is exported and included.
Working prototype	A complete vertical slice: real child input produces real AI output which produces real stored state.
Innovation	Adaptive difficulty driven by effort rather than correctness, plus interpersonal skills that no competitor measures.
Safety and responsibility	Moderation gate before evaluation, Row-Level Security, private storage, and deliberate PII minimisation.
Presentation	Layered architecture diagrams and a scripted five-minute demo with a live automation trigger.

12. Day-by-Day Execution Plan

Twelve working days from a standing start. Compress the frontend phase if you are tighter than this, never the Postman phase.

Day	Focus	Deliverable at end of day
1	Scope lock + accounts + 1-pager	All five tools logged in; 1-pager written
2	Supabase schema + RLS + storage	Nine tables live, RLS green on all
3	Postman: Mission Generator	Six valid missions across three pillars
4	Postman: Evaluator + Moderation	All four tone tests passing
5	Edge Function: generate-mission	Endpoint deployed and returning missions
6	Edge Function: submit-response	Full loop working in Postman
7	Edge Function: get-progress	Collection runs end to end in one click
8	Lovable: onboarding + mission screen	Child can complete a mission in a browser
9	Lovable: parent dashboard + polish	All three screens working on mobile width
10	Make.com digest + seed demo data	Digest email received in inbox
11	Testing, security checks, bug fixes	Both checklists fully passed
12	Record demo, write Notion docs, post	Everything submitted

12.1 Risk register

Risk	Likelihood	Mitigation
AI returns unparseable JSON	Medium	JSON mode plus try/catch with one retry, then a hardcoded fallback mission
Lovable cannot wire a custom endpoint	Medium	Keep a plain HTML + fetch page as a working fallback; function it, then pretty it
Evaluation feels slow on camera	High	Design the “thinking” animation deliberately; it converts latency into personality
Running out of time on the frontend	High	Screens 1 and 2 are mandatory; the dashboard can be a single scrolling page
OpenAI cost overrun	Low	Hard spend limit plus the daily mission cache
Demo fails live	Medium	Record the video in advance and present the recording; never demo live without a backup

13. Appendix

13.1 API contract summary

Endpoint	Method	Request	Response
/functions/v1/generate-mission	POST	{ child_id, pillar }	Full mission row
/functions/v1/submit-response	POST	{ mission_id, child_id, answer_text, media_path, time_spent_sec }	{ feedback, badge, new_difficulty }
/functions/v1/get-progress	GET	?child_id=uuid	{ child, pillars, badges, recent_insights, latest_digest }

13.2 Environment variables

Variable	Lives in	Exposed to browser
OPENAI_API_KEY	Edge Function secrets	Never
SERVICE_ROLE_KEY	Edge Function secrets	Never
SUPABASE_URL	Both frontend and functions	Yes
SUPABASE_ANON_KEY	Frontend	Yes

13.3 Badge award rules

Badge code	Awarded when
first_steps	The child's first evaluation row is written
streak_3	skill_profile.streak_days reaches 3 in any pillar
streak_7	skill_profile.streak_days reaches 7 in any pillar
logic_master	Five LEARN evaluations with a logic score of 4 or above
story_weaver	Three completed CREATE missions with input_type text
kind_heart	Three completed CONNECT missions
level_up	skill_profile.difficulty reaches 4 in any pillar

13.4 Glossary

Term	Plain meaning
Edge Function	A small piece of server code that runs on request. Used here because it can hold secrets safely, unlike browser code.
Row-Level Security	A database rule that decides, row by row, who is allowed to see what. Enforced by the database itself, so a bug in the app cannot bypass it.
JSON mode	An OpenAI setting that forces the model to return strictly valid JSON, so your code can read it reliably.
Signed URL	A temporary web address for a private file. It expires, so it cannot be shared or reused.
Idempotent	An operation you can safely run twice without causing a duplicate. Used here for badge awards and weekly digests.
Vertical slice	One complete path through every layer of the system — proof the whole thing connects before anything is polished.